

Unidad Didáctica

**Factibilidad del Proyecto de
Software, Unidad III**

Contenido

Índice de Contenido	2
Introducción	3
Desarrollo de Contenidos	4
Búsqueda de información para la definición del problema	4
Presentación gráfica de Información recopilada	12
Determinación de la viabilidad	19
Estudio de factibilidad.	19
Técnica	19
Económica	21
Operativa	23
Legal	24
Planeación y control de actividades	26
Diagramas de Gantt.....	26
Diagramas de Pert	26
Bibliografía y Webgrafía	27
Glosario	27



Bienvenidos a la tercera unidad de nuestro curso sobre Factibilidad del Proyecto de Software. En esta etapa, nos adentraremos en la evaluación exhaustiva de la viabilidad de un proyecto propuesto, centrándonos en la definición precisa de un problema de negocios y su correspondiente análisis para determinar la factibilidad del proyecto en cuestión.

Nuestros objetivos principales serán definir de manera clara y concisa el problema empresarial que busca solucionar el proyecto propuesto, así como también escribir y presentar una propuesta de estudio de factibilidad de sistemas efectiva, donde se enfatizará tanto en el análisis como en el diseño para garantizar su viabilidad y éxito.

Para lograr estos objetivos, exploraremos diversas etapas y técnicas fundamentales. Comenzaremos con la búsqueda meticulosa de información relevante para la definición del problema, seguida de la presentación gráfica de la información recopilada para una mejor comprensión y análisis.

Posteriormente, nos adentraremos en la fase crucial de determinar la viabilidad del proyecto a través de un estudio exhaustivo, abarcando aspectos técnicos, económicos, operativos y legales. Este análisis integral nos proporcionará una visión clara sobre la factibilidad del proyecto propuesto.

Además, abordaremos la planificación y el control de actividades necesarias para llevar a cabo el proyecto con éxito, utilizando herramientas como los Diagramas de Gantt y los Diagramas de Pert para una gestión eficiente del tiempo y los recursos.

Búsqueda de información para la definición del problema

La búsqueda de información para la definición del problema es una etapa crítica en cualquier proyecto de desarrollo de software. A través de esta, se asegura que el problema a resolver esté claramente identificado y comprendido, lo que es esencial para desarrollar una solución adecuada. A continuación, se describen diversos métodos y técnicas para recopilar la información necesaria y se proporcionan indicaciones de cómo utilizar esta información para la redacción del problema.

Encuestas

Las encuestas son herramientas valiosas para recopilar información de un gran número de personas de manera eficiente. En el contexto del desarrollo de software, pueden ser utilizadas para entender las necesidades y expectativas de los usuarios finales, identificar problemas existentes y evaluar el interés en nuevas funcionalidades.

Ventajas:

- Alcance amplio: Permiten recopilar datos de muchas personas rápidamente.
- Estandarización: Preguntas uniformes facilitan el análisis de datos.

Desventajas:

- Respuestas superficiales: La información puede ser limitada en detalle.
- Tasa de respuesta: Puede ser baja si los encuestados no están motivados.

Cómo usarlas

- **Diseño de la encuesta:** Formule preguntas claras y concisas, tanto abiertas como cerradas, para captar una amplia gama de respuestas. Ejemplo: "¿Qué problemas enfrenta actualmente con el sistema actual?" o "¿Qué funcionalidades adicionales consideraría útiles en una nueva versión del software?"
- **Distribución y recopilación:** Utilice plataformas online como Google Forms o SurveyMonkey para distribuir la encuesta y recopilar respuestas. Asegúrese de enviar recordatorios para aumentar la tasa de respuesta.

Ejemplo: Una encuesta podría preguntar a los empleados de una empresa sobre las dificultades que enfrentan con el software actual, así como las características que les gustaría ver en una nueva versión.

Redacción del problema: Analice las respuestas para identificar tendencias y problemas comunes. Por ejemplo, si un gran número de encuestados menciona dificultades con la interfaz de usuario, esto se debe reflejar en la definición del problema: "Los usuarios encuentran la interfaz del software actual poco intuitiva, lo que dificulta su uso eficiente."

Listas de Cotejo

Las listas de cotejo son herramientas estructuradas que permiten verificar la presencia o ausencia de características específicas en el contexto del problema.

Ventajas:

- Claridad: Facilitan la identificación de elementos específicos.
- Simplicidad: Son fáciles de diseñar y utilizar.

Desventajas:

- Limitación en profundidad: Pueden no capturar información detallada sobre cada ítem.

Cómo usarlas:

- Elaboración de la lista: Incluya aspectos clave que necesita verificar. Ejemplo: "¿El sistema permite generar informes personalizados?" o "¿El software se integra con otras herramientas de la empresa?"
- Aplicación de la lista: Utilice la lista durante reuniones con usuarios clave o al revisar documentación del sistema actual.

Ejemplo: Una lista de cotejo podría incluir elementos como "funcionalidades actuales que no se utilizan" o "errores comunes reportados por los usuarios".

Redacción del problema: Identifique y documente los ítems faltantes o problemáticos. Por ejemplo: "El sistema actual carece de capacidad para generar informes personalizados, lo cual es una necesidad crítica para los usuarios."

Guías de Observación

Las guías de observación permiten a los investigadores observar directamente cómo se lleva a cabo una tarea o cómo se utiliza un sistema, proporcionando información valiosa sobre problemas y oportunidades de mejora.

Ventajas:

- Información en contexto: Permite ver el uso del software en su entorno natural.
- Datos cualitativos ricos: Captura detalles que otros métodos pueden pasar por alto.

Desventajas:

- Tiempo intensivo: Puede requerir mucho tiempo para observar y analizar.
- Sesgo del observador: La presencia del observador puede influir en el comportamiento de los usuarios.

Cómo usarlas:

- Preparación de la guía: Defina los aspectos a observar. Ejemplo: "Tareas repetitivas realizadas con el software" o "Puntos donde los usuarios solicitan ayuda."
- Realización de la observación: Observe y tome notas detalladas sobre el uso del software en situaciones reales. Capture tanto comportamientos exitosos como dificultades encontradas.

Ejemplo: Observar a los empleados mientras usan un sistema de gestión de inventarios para identificar cuellos de botella y áreas donde el software podría mejorarse.

Redacción del problema: Documente los problemas observados con ejemplos específicos. Por ejemplo: "Los usuarios encuentran dificultades para localizar funciones específicas, lo que ralentiza el proceso de gestión de inventarios."

Entrevistas

Las entrevistas permiten obtener información detallada y profunda de los actores clave del proyecto. Pueden ser estructuradas, semiestructuradas o no estructuradas.

Ventajas:

- Profundidad de información: Permiten explorar temas en detalle.
- Flexibilidad: Se pueden adaptar en tiempo real según las respuestas del entrevistado.

Desventajas:

- Tiempo intensivo: Tanto en la realización como en el análisis de las entrevistas.
- Sesgo del entrevistador: Las preguntas y la interacción pueden influir en las respuestas

Cómo usarlas:

- Preparación de la entrevista: Desarrolle una guía con preguntas clave, pero deje espacio para explorar temas emergentes. Ejemplo: "¿Qué características del software actual encuentra más útiles y por qué?" o "¿Qué cambios sugeriría para mejorar su experiencia?"
- Conducción de la entrevista: Realice entrevistas en un entorno cómodo para el entrevistado, asegurándose de escuchar activamente y tomar notas detalladas.

Ejemplo: Entrevistar a los usuarios principales del sistema para comprender sus necesidades específicas, frustraciones y deseos para la nueva versión del software.

Redacción del problema: Utilice las transcripciones para identificar temas recurrentes y preocupaciones. Por ejemplo: "Los usuarios expresan frustración con la velocidad de respuesta del sistema durante picos de uso."

Benchmarking

El benchmarking implica la investigación de aplicaciones similares para identificar mejores prácticas, características útiles y estándares de la industria que puedan aplicarse al proyecto.

Ventajas:

- Identificación de mejores prácticas: Aprender de lo que ya ha funcionado en otros contextos.
- Comparación directa: Permite comparar el proyecto con estándares de la industria.

Desventajas:

- Relevancia variable: Las soluciones de otros contextos pueden no ser completamente aplicables.
- Costo y tiempo: Puede ser costoso y llevar tiempo realizar una investigación exhaustiva.

Cómo usarlas:

- Selección de aplicaciones para benchmarking: Elija aplicaciones líderes en el mercado que cumplan funciones similares. Ejemplo: "Analizar herramientas de gestión de proyectos como Trello o Asana."
- Análisis de características: Compare funcionalidades, usabilidad y desempeño con el sistema actual o proyectado.

Ejemplo: Analizar aplicaciones de gestión de proyectos ya existentes para identificar funcionalidades clave y áreas de mejora que se pueden incorporar en el nuevo software.

Redacción del problema: Documente las diferencias y las áreas donde el software actual queda corto en comparación con las mejores prácticas. Por ejemplo: "El software actual no soporta colaboración en tiempo real, una característica común en otras herramientas de gestión de proyectos."

Análisis de Documentación Existente

El análisis de documentación incluye la revisión de manuales, informes, registros y cualquier otra documentación relevante que pueda proporcionar información sobre el problema.

Ventajas:

- Información detallada: Proporciona contexto histórico y detalles específicos.
- Facilidad de acceso: Generalmente está disponible dentro de la organización.

Desventajas:

- Información desactualizada: La documentación puede no reflejar la situación actual.
- Volumen de datos: Puede ser abrumador revisar una gran cantidad de documentos.

Cómo usarlas:

- Revisión sistemática: Examine documentos relevantes como reportes de incidentes, manuales de usuario y registros de uso. Busque patrones y recurrencias en los problemas reportados.
- Síntesis de información: Resuma los hallazgos de manera estructurada, identificando las áreas problemáticas.

Ejemplo: Revisar los registros de incidentes del sistema actual para identificar patrones de fallos y áreas recurrentes de problemas.

Redacción del problema: Utilice los datos para respaldar la identificación del problema. Por ejemplo: "Los registros de incidentes muestran que el sistema falla con frecuencia durante las operaciones de carga masiva de datos."

Implementación en Metodologías Ágiles (Scrum)

En un entorno ágil, el Scrum Master puede facilitar la recopilación de información organizando sesiones de recopilación de requisitos durante las reuniones de planificación del sprint. Se pueden utilizar técnicas como:

- Reuniones de inicio de Sprint (Sprint Planning): Definir y priorizar las tareas de recopilación de información: Asigne actividades de recopilación de datos como entrevistas y encuestas a los miembros del equipo según su disponibilidad y habilidades.
- Establecer objetivos claros: Asegúrese de que cada tarea tenga un propósito claro y esté alineada con la definición del problema.
- Reuniones diarias (Daily Stand-ups): Monitorear el progreso: Cada miembro del equipo informa sobre el estado de sus tareas de recopilación de información, permitiendo ajustes rápidos si surgen impedimentos.
- Resolver impedimentos: El Scrum Master ayuda a resolver cualquier problema que pueda estar bloqueando la recopilación de información.
- Revisión y Retrospectiva del Sprint (Sprint Review and Retrospective): Evaluar la información recopilada: Revise los datos obtenidos y discútalos en equipo para asegurarse de que todos tengan una comprensión clara del problema.
- Ajustar y mejorar: Reflexione sobre el proceso de recopilación de información para identificar mejoras que pueden implementarse en el próximo sprint.

La combinación de estos métodos proporciona una base sólida para la definición precisa del problema, lo cual es esencial para el éxito del proyecto de software. El uso de metodologías ágiles asegura que la información se recopile de manera continua y se adapte según las necesidades del proyecto, facilitando la toma de decisiones informadas y oportunas.

Presentación gráfica de Información recopilada

Una vez recopilada la información para la definición del problema, es esencial presentarla de manera clara y comprensible. La presentación gráfica de datos ayuda a visualizar patrones, tendencias y relaciones que pueden no ser evidentes en formato de texto. A continuación, se describen diversas técnicas y herramientas para la presentación gráfica de información recopilada, con indicaciones sobre cómo utilizarlas de manera efectiva.

Diagramas de Barras

Los diagramas de barras son ideales para comparar cantidades entre diferentes grupos o categorías. Son útiles para visualizar datos categóricos y resaltar diferencias significativas.

Ventajas:

- Claridad: Facilitan la comparación directa entre diferentes categorías.
- Simplicidad: Son fáciles de entender y crear.

Desventajas:

- Limitación en detalles: No muestran información detallada dentro de cada categoría.

Cómo usarlas: Seleccionar datos relevantes: Identifique las categorías clave y las cantidades a comparar. Ejemplo: "Número de incidencias reportadas por tipo de error."

Crear el diagrama: Utilice herramientas como Excel, Google Sheets, o software de visualización de datos como Tableau o Power BI.

Ejemplo: Un diagrama de barras podría mostrar la frecuencia de diferentes tipos de errores reportados en el sistema actual.

Interpretación: Analice las barras para identificar los tipos de errores más frecuentes. Esto puede resaltar áreas críticas que necesitan atención inmediata.

Gráficos de Líneas

Los gráficos de líneas son útiles para mostrar tendencias a lo largo del tiempo, ideal para datos continuos. Ayudan a visualizar cambios y patrones de comportamiento.

Ventajas:

- Visualización de tendencias: Facilitan la observación de cómo cambian los datos a lo largo del tiempo.
- Detección de patrones: Ayudan a identificar tendencias

ascendentes, descendentes o cíclicas.

Desventajas:

- Complejidad en datos múltiples: Pueden volverse difíciles de leer si se incluyen demasiadas líneas.

Cómo usarlas: Recolección de datos temporales: Reúna datos que varíen con el tiempo. Ejemplo: "Número de incidencias por mes durante el último año."

Crear el gráfico: Utilice herramientas de gráficos como Excel, Google Sheets, o software especializado en visualización de datos.

Ejemplo: Un gráfico de líneas podría mostrar la cantidad de incidencias reportadas cada mes a lo largo del año.

Interpretación: Identifique picos o disminuciones en las incidencias. Esto puede indicar periodos problemáticos que requieren investigación adicional.

Diagramas de Pastel

Los diagramas de pastel son efectivos para mostrar proporciones y distribuciones dentro de un conjunto de datos. Son ideales para visualizar partes de un todo.

Ventajas:

- Visualización de proporciones: Muestran claramente cómo se distribuyen las partes de un todo.
- Simplicidad: Son fáciles de interpretar a primera vista.

Desventajas:

- Limitación en precisión: No son ideales para comparar pequeñas diferencias entre categorías.
- Dificultad con muchos segmentos: Pueden volverse confusos si hay demasiados segmentos.

Cómo usarlas: Selección de categorías: Identifique las categorías y sus proporciones dentro del total. Ejemplo: "Distribución de tipos de usuarios en el sistema."

Crear el diagrama: Utilice herramientas de gráficos como Excel, Google Sheets, o software de visualización.

Ejemplo: Un diagrama de pastel podría mostrar la proporción de usuarios administrativos, usuarios operativos y usuarios invitados en el sistema.

Interpretación: Analice las proporciones para comprender mejor la composición del grupo de usuarios, lo que puede influir en las decisiones de diseño del sistema.

Diagramas de Dispersión

Los diagramas de dispersión son útiles para mostrar relaciones y correlaciones entre dos variables. Son ideales para identificar patrones y outliers.

Ventajas:

- Visualización de relaciones: Ayudan a identificar correlaciones entre variables.
- Detección de outliers: Facilitan la identificación de valores atípicos.

Desventajas:

- Interpretación subjetiva: Requieren una interpretación cuidadosa para no confundir correlación con causalidad.

Cómo usarlas: Recolección de datos variables: Reúna pares de datos que se puedan correlacionar. Ejemplo: "Tiempo de respuesta del sistema vs. número de usuarios simultáneos."

Crear el diagrama: Utilice herramientas de gráficos como Excel, Google Sheets, o software especializado.

Ejemplo: Un diagrama de dispersión podría mostrar la relación entre el tiempo de respuesta del sistema y el número de usuarios simultáneos.

Interpretación: Identifique si hay una correlación entre las variables, lo que puede indicar que el rendimiento del sistema disminuye con un mayor número de usuarios.

Mapas de Calor

Los mapas de calor son efectivos para mostrar la densidad de valores en un espacio bidimensional, destacando áreas de alta y baja concentración.

Ventajas:

- Visualización de densidad: Ayudan a identificar áreas de alta y baja concentración.
- Fácil identificación de patrones: Destacan rápidamente las áreas problemáticas.

Desventajas:

- Interpretación compleja: Pueden requerir una explicación adicional para algunos públicos.

Cómo usarlas: Recolección de datos espaciales: Reúna datos que se puedan representar en una matriz bidimensional. Ejemplo: "Frecuencia de uso de funcionalidades en diferentes módulos del sistema."

Crear el mapa de calor: Utilice software especializado en visualización de datos como Tableau, Power BI o herramientas de programación como Python con librerías como Seaborn o Matplotlib.

Ejemplo: Un mapa de calor podría mostrar la frecuencia de uso de diferentes funcionalidades en el sistema, destacando cuáles son más y menos utilizadas.

Interpretación: Analice las áreas de alta y baja concentración para identificar funcionalidades críticas y las que requieren mejoras o pueden ser eliminadas.

Tablas y Tablas Dinámicas

Las tablas y tablas dinámicas son herramientas versátiles para organizar y resumir grandes cantidades de datos. Son útiles para presentar datos detallados y hacer análisis más profundos.

Ventajas:

- Organización detallada: Permiten desglosar datos en detalles específicos.
- Flexibilidad: Las tablas dinámicas permiten reorganizar

y resumir datos de múltiples formas.

Desventajas:

- Menos visuales: Pueden ser menos atractivas y más difíciles de interpretar a primera vista en comparación con gráficos.

Cómo usarlas: Recolección de datos estructurados: Compile datos en un formato tabular. Ejemplo: "Listado de incidencias con detalles de fecha, tipo de error, y resolución."

Crear la tabla o tabla dinámica: Utilice herramientas como Excel o Google Sheets. Las tablas dinámicas permiten filtrar y reorganizar datos fácilmente para análisis detallados.

Ejemplo: Una tabla dinámica podría resumir el número de incidencias por tipo de error y mes, proporcionando una vista detallada y organizada.

Interpretación: Use las tablas para identificar patrones específicos y realizar análisis detallados, como determinar qué tipos de errores son más frecuentes en ciertos periodos.

Implementación en Metodologías Ágiles (Scrum)

En un entorno ágil, el Scrum Master y el equipo de desarrollo pueden utilizar estas herramientas para visualizar y analizar datos de manera continua durante los sprints.

Aquí hay algunas formas de integrar estas técnicas en el proceso ágil:

- Reuniones de inicio de Sprint (Sprint Planning): Definir objetivos de visualización: Establezca qué tipos de gráficos se necesitan para visualizar la información recopilada.
- Asignación de tareas: Asigne la creación de gráficos y el análisis de datos a miembros específicos del equipo.
- Reuniones diarias (Daily Stand-ups): Monitoreo de progreso: Revise el progreso en la creación y análisis de gráficos. Resuelva impedimentos rápidamente.
- Actualización de gráficos: Actualice los gráficos con nuevos datos conforme se recopilen durante el sprint.
- Revisión y Retrospectiva del Sprint (Sprint Review and Retrospective): Presentación de resultados: Utilice gráficos para presentar hallazgos y analizar el desempeño del sprint.
- Mejoras continuas: Discuta cómo los gráficos ayudaron en la toma de decisiones y cómo mejorar el proceso de visualización de datos en futuros sprints.

La presentación gráfica de información recopilada facilita la comprensión y análisis de datos, permitiendo a los equipos de desarrollo de software tomar decisiones informadas y basadas en evidencia. Utilizar metodologías ágiles asegura que estos gráficos se actualicen y utilicen de manera continua y efectiva, mejorando la calidad y éxito del proyecto de software.

Determinación de la viabilidad

La determinación de la viabilidad es un paso crucial en la evaluación de un proyecto de software. Consiste en analizar diversos aspectos del proyecto para asegurarse de que es factible desde diferentes perspectivas. Estos aspectos incluyen la viabilidad técnica, económica, operativa, legal y de calendario. A continuación, se detallan cada uno de estos aspectos, junto con metodologías y herramientas para evaluarlos, y se proporciona una guía sobre cómo usar esta información para tomar decisiones informadas.

Estudio de factibilidad.

Técnica

La factibilidad técnica de un proyecto de desarrollo de software es crucial para su viabilidad y éxito. Implica evaluar si se cuentan con los recursos técnicos y tecnológicos necesarios para implementar y ejecutar el proyecto de manera efectiva. Aquí se detallan los aspectos clave que deben considerarse en esta evaluación:

- **Evaluación de Equipos Actuales:** Inicia evaluando las condiciones de los equipos de hardware y software existentes en la empresa. Esta revisión incluye verificar la capacidad de procesamiento, memoria, almacenamiento y otros recursos, así como la versión y estado de los sistemas operativos y aplicaciones instaladas. Esta evaluación proporciona una visión clara de los recursos disponibles y posibles limitaciones técnicas.
- **Requerimientos Mínimos y Óptimos:** Identifica y especifica los requerimientos mínimos y óptimos para las tecnologías a utilizar, desde el sistema operativo hasta la base de datos y las herramientas de desarrollo. Es importante considerar las clases de Sistemas Operativos y Administración de Bases de Datos para determinar estos requisitos de manera precisa.
- **Evaluación de Infraestructura:** Determina si se necesita implementar una infraestructura adicional, como servidores, redes o sistemas de

almacenamiento. Evalúa la capacidad y disponibilidad de la infraestructura existente para asegurar que pueda soportar los requisitos del proyecto, considerando posibles actualizaciones o mejoras necesarias.

- **Virtualización y Cloud Computing:** Considera la viabilidad de utilizar tecnologías de virtualización o cloud computing. Evalúa los beneficios y desafíos de la virtualización de servidores, la implementación de máquinas virtuales o el uso de servicios en la nube como AWS, Azure o Google Cloud.
- **Integración de Sistemas:** Analiza la necesidad de integrar el nuevo sistema con sistemas existentes en la empresa. Esto puede implicar el desarrollo de interfaces de integración, migración de datos o sincronización de sistemas. Evalúa la compatibilidad y capacidad de integración de los sistemas existentes con el nuevo sistema.
- **Capacidades de Seguridad:** Evalúa las capacidades de seguridad necesarias para proteger el sistema y los datos. Esto incluye la implementación de firewalls, sistemas de detección de intrusiones o políticas de acceso. Verifica si los recursos técnicos existentes son adecuados para garantizar la seguridad del sistema.
- **Documentación del Estudio:** Prepara un informe detallado que documente todos los hallazgos, conclusiones y recomendaciones del estudio. Presenta el informe a los stakeholders para obtener su aprobación y feedback, asegurando su validez y alineación con los objetivos del proyecto.
- **Diseño de la Arquitectura del Sistema:** Crea un diseño detallado que incluya diagramas y especificaciones técnicas para garantizar la viabilidad y coherencia del sistema. Realiza revisiones por pares y valida el diseño con expertos para asegurar su eficacia.
- **Benchmarking:** Compara tecnologías y revisa estudios de casos de proyectos similares para seleccionar las soluciones más adecuadas y aprender de las experiencias pasadas.

La aplicación de Scrum en la evaluación de factibilidad técnica permite un enfoque colaborativo y adaptativo, asegurando una evaluación exhaustiva y eficiente de los aspectos técnicos del proyecto. Esto facilita la toma de decisiones informadas y la identificación temprana de posibles desafíos técnicos, contribuyendo al éxito general del proyecto.

Económica

La factibilidad económica de un proyecto de desarrollo de software es crucial para determinar si es viable desde el punto de vista financiero. Aquí tienes una descripción detallada de los aspectos que debes considerar:

- **Costos de Desarrollo:** Se deben calcular todos los costos asociados con el desarrollo del software, que incluyen salarios del equipo de desarrollo, costos de herramientas y tecnologías, costos de infraestructura (hardware, software), gastos de licencias, costos de capacitación, entre otros. Es importante estimar estos costos de manera realista y tener en cuenta posibles gastos adicionales que puedan surgir durante el desarrollo.
- **Costos de Mantenimiento:** Además de los costos de desarrollo inicial, también debes considerar los costos de mantenimiento del software a lo largo de su ciclo de vida. Esto incluye actualizaciones, correcciones de errores, soporte técnico, alojamiento web, entre otros. Estos costos pueden ser significativos a largo plazo y deben ser tenidos en cuenta en el análisis de factibilidad económica.
- **Beneficios y Ganancias Esperadas:** Debes identificar y cuantificar los beneficios y ganancias esperadas que se obtendrán a través del desarrollo del software. Esto puede incluir aumento de ingresos, reducción de costos operativos, acceso a nuevos mercados, mejora de la eficiencia, entre otros. Es importante realizar proyecciones realistas y considerar diferentes escenarios para evaluar el potencial de retorno de la inversión.

- **Análisis de Costo-Beneficio:** El análisis de costo-beneficio compara los costos totales del proyecto con los beneficios esperados para determinar si la inversión es justificable. Debes calcular el valor presente neto (VPN), la tasa interna de retorno (TIR), el período de recuperación de la inversión (PRI) y otros indicadores financieros relevantes. Esto te ayudará a evaluar la rentabilidad del proyecto y tomar decisiones informadas sobre su viabilidad económica.
- **Consideración de Riesgos:** Debes identificar y evaluar los riesgos financieros asociados con el proyecto, como cambios en los costos, retrasos en el desarrollo, cambios en la demanda del mercado, competencia, entre otros. Es importante tener en cuenta estos riesgos al realizar el análisis de factibilidad económica y desarrollar estrategias de mitigación para reducir su impacto en el proyecto.
- **Alternativas de Financiamiento:** Si el proyecto requiere financiamiento externo, debes explorar diferentes opciones de financiamiento, como inversores, préstamos bancarios, financiamiento gubernamental, capital de riesgo, entre otros. Debes evaluar las condiciones y términos de cada opción y determinar cuál es la más adecuada para tus necesidades y circunstancias.

Al realizar un análisis exhaustivo de estos aspectos, podrás determinar si el proyecto de desarrollo de software es económicamente factible y tomar decisiones informadas sobre su implementación. Es importante revisar y actualizar regularmente el análisis económico a lo largo del ciclo de vida del proyecto para adaptarse a cambios en las condiciones del mercado y minimizar riesgos financieros.

Además se debe tener en cuenta el estado actual del negocio, cantidad de empleados, si se necesita capacitar a dichos empleados, lo cual implica un coste económico y de tiempo, si se necesitan contratar empleados, y se debe crear un presupuesto

Operativa

La factibilidad operativa de un proyecto de desarrollo de software se refiere a la capacidad del proyecto para ser implementado y ejecutado con éxito en términos de operaciones y procesos. Aquí tienes una descripción detallada de los aspectos que debes considerar:

- Capacidad Organizacional: Evalúa la capacidad de la organización para llevar a cabo el proyecto de desarrollo de software. Esto incluye tener el personal adecuado con las habilidades y experiencia necesarias, así como los recursos y la infraestructura disponibles para apoyar el desarrollo y la implementación del software. Aquí se vinculará la clase de Principios de administración
- Procesos y Procedimientos: Verifica si la organización cuenta con procesos y procedimientos establecidos que puedan respaldar el desarrollo de software de manera eficiente y efectiva. Esto puede incluir metodologías de desarrollo de software, estándares de calidad, prácticas de gestión de proyectos, entre otros.
- Disponibilidad de Recursos: Asegúrate de que se disponga de los recursos necesarios para llevar a cabo el proyecto, como hardware, software, herramientas de desarrollo, equipos de trabajo, entre otros. Además, debes considerar si estos recursos estarán disponibles en el momento adecuado y en la cantidad suficiente para cumplir con los objetivos del proyecto.
- Capacidades Tecnológicas: Evalúa si la organización cuenta con las capacidades tecnológicas necesarias para desarrollar y mantener el software. Esto incluye tener acceso a tecnologías y plataformas adecuadas, así como la capacidad de integrar el software con los sistemas existentes de la organización.
- Cambios Organizacionales: Considera si el proyecto requerirá cambios significativos en la organización, como la reestructuración de equipos, la implementación de nuevos procesos o la adopción de nuevas tecnologías. Debes evaluar la capacidad de la organización para gestionar y adaptarse a estos cambios de manera efectiva. Principios de Administración

- Capacitación y Soporte: Asegúrate de que se proporcione la capacitación adecuada al personal involucrado en el proyecto para garantizar que estén preparados para cumplir con sus roles y responsabilidades. Además, debes asegurar que se proporcione un soporte adecuado durante y después de la implementación del software para abordar cualquier problema o desafío que pueda surgir.
- Aceptación de Usuarios: Verifica si los usuarios finales del software están dispuestos y preparados para adoptar y utilizar el sistema. Esto puede requerir la realización de actividades de sensibilización, demostraciones de software, pruebas de usuario, entre otros, para garantizar la aceptación y adopción del software por parte de los usuarios.

Al evaluar estos aspectos de la factibilidad operativa, podrás determinar si el proyecto de desarrollo de software puede ser implementado y ejecutado de manera efectiva dentro de la organización. Es importante identificar y abordar cualquier desafío operativo potencial desde el principio para garantizar el éxito del proyecto.

Legal

El estudio de factibilidad legal es crucial para determinar si un proyecto de desarrollo de software es viable desde el punto de vista legal y regulatorio. Aquí se detallan los aspectos clave que deben considerarse en este análisis:

- Cumplimiento Normativo: Evalúa si el proyecto de software cumple con todas las leyes, regulaciones y normativas aplicables en el ámbito legal. Esto puede incluir leyes de protección de datos, propiedad intelectual, privacidad, seguridad de la información y cualquier otra normativa relevante para el desarrollo y uso del software.
- Licencias y Derechos de Autor: Verifica que el software desarrollado cumpla con las licencias de software y los derechos de autor correspondientes. Esto implica asegurarse de que se respeten los términos y condiciones de las

licencias de software de terceros utilizadas en el proyecto, así como garantizar la protección de los derechos de autor del software desarrollado internamente.

- Protección de Datos Personales: Asegúrate de que el proyecto cumpla con las leyes de protección de datos personales aplicables. Esto incluye el cumplimiento de regulaciones como el Reglamento General de Protección de Datos (GDPR) en la Unión Europea o leyes similares en otras jurisdicciones, garantizando la privacidad y seguridad de los datos personales recopilados y procesados por el software.
- Seguridad de la Información: Evalúa las medidas de seguridad implementadas en el software para proteger la información confidencial y sensible. Esto incluye la implementación de controles de acceso, cifrado de datos, medidas de prevención de intrusiones y cualquier otra medida necesaria para garantizar la seguridad de la información.
- Propiedad Intelectual: Verifica que el proyecto respete los derechos de propiedad intelectual de terceros y proteja los activos de propiedad intelectual propios. Esto implica realizar búsquedas de patentes y marcas registradas para evitar infringir los derechos de propiedad intelectual de terceros y proteger los derechos de propiedad intelectual del software desarrollado.
- Contratos y Acuerdos: Asegúrate de que se establezcan contratos y acuerdos claros y completos con todas las partes involucradas en el proyecto, incluidos clientes, proveedores, contratistas y empleados. Esto ayuda a definir claramente los derechos y responsabilidades de cada parte y minimiza el riesgo de conflictos legales en el futuro.

Al realizar un análisis exhaustivo de estos aspectos, se puede determinar si el proyecto de desarrollo de software es legalmente factible y tomar las medidas necesarias para garantizar el cumplimiento de todas las leyes y regulaciones aplicables. Esto ayuda a minimizar el riesgo de enfrentar problemas legales y a asegurar el éxito del proyecto en el largo plazo.

Planeación y control de actividades

En esta sección, nos adentraremos en la elaboración de un plan detallado que abarque todas las actividades necesarias para completar el proyecto de desarrollo de software, utilizando metodologías ágiles con enfoque Scrum. Esto implica la identificación de tareas específicas, asignación de recursos y establecimiento de plazos realistas para cada una, todo ello bajo el marco de trabajo ágil.

Diagramas de Gantt

El Diagrama de Gantt es una herramienta útil para visualizar la secuencia de actividades y la duración de cada una en un proyecto de desarrollo de software. En el contexto de Scrum, podemos adaptar esta herramienta para mostrar la planificación de los Sprints y la distribución de las tareas a lo largo del tiempo. Cada Sprint se representaría como una barra en el diagrama, con las tareas asociadas dentro de cada Sprint. Esto proporciona una visión clara del progreso del proyecto y ayuda a identificar posibles desviaciones en el plan.

Diagramas de Pert

El Diagrama PERT (Program Evaluation and Review Technique) es otra herramienta valiosa que podemos utilizar en la planificación y control de actividades en un proyecto de desarrollo de software. A diferencia del Diagrama de Gantt, el Diagrama PERT se centra en identificar las dependencias entre las tareas y calcular la ruta crítica del proyecto. En Scrum, esto nos permite visualizar las interdependencias entre las diferentes funcionalidades y determinar las tareas críticas que deben completarse para cumplir con los objetivos del proyecto en el tiempo previsto.

Es fundamental monitorear regularmente el progreso del proyecto y ajustar el plan según sea necesario para garantizar que se cumplan los objetivos dentro del plazo establecido. La metodología Scrum proporciona mecanismos integrados para la

inspección y adaptación continua, como las reuniones diarias de Scrum, la revisión del Sprint y la retrospectiva del Sprint. Estas prácticas ágiles permiten una gestión flexible y eficiente del proyecto, maximizando el valor entregado al cliente en cada iteración.

Bibliografía y Webgrafía

- Montilva, J. C., & Barrios, J. A. (2021). **Ingeniería del Software: Un enfoque basado en procesos** (Edición en Español). Universidad de Los Andes.
- Martel, A. (2014). **Gestión práctica de proyectos con Scrum: Desarrollo de software ágil para el Scrum Master** (Aprender a ser mejor gestor de proyectos) (3rd ed., Edición en Español). CreateSpace Independent Publishing Platform.
- Simões, G. S., & Vazquez, C. E. (2018). **Ingeniería de Requisitos: Software Orientado al Negocio** (Spanish Edition). Independently published.
- Pressman, R. S. (2010). **Ingeniería de Software** (Spanish Edition) (7th ed.). McGraw-Hill Interamericana Editores S.A. de C.V.
- McCarthy, R. (2020). **Agile y Scrum: Descubra el poder de la gestión de proyectos Agile, Lean Thinking, el proceso Kanban y Scrum** (Spanish Edition). Independently published.

- **Cumplimiento Normativo:** Evaluación del grado en que un proyecto cumple con las leyes, regulaciones y normativas aplicables en un determinado ámbito legal.
- **Diagrama de Gantt:** Herramienta utilizada para visualizar la secuencia de actividades y la duración de cada una en un proyecto, mostrando las fechas de inicio y finalización de cada tarea.
- **Diagrama PERT:** Técnica de gestión de proyectos que se utiliza para representar y analizar las tareas y las dependencias entre ellas, calculando la ruta crítica del proyecto.
- **Factibilidad del Proyecto de Software:** Evaluación de la viabilidad de un proyecto de desarrollo de software desde diferentes perspectivas, como técnica, económica, operativa y legal.
- **Factibilidad Económica:** Análisis de la viabilidad financiera de un proyecto de desarrollo de software, considerando costos de desarrollo, mantenimiento, beneficios esperados, análisis de costo-beneficio, riesgos financieros y alternativas de financiamiento.
- **Factibilidad Legal:** Evaluación de la viabilidad legal y regulatoria de un proyecto de desarrollo de software, considerando aspectos como cumplimiento normativo, licencias de software, protección de datos personales, seguridad de la información, propiedad intelectual y contratos legales.
- **Factibilidad Operativa:** Análisis de la viabilidad operativa de un proyecto de desarrollo de software, considerando requisitos operativos, disponibilidad de recursos, capacidades organizacionales, planificación y programación, gestión del cambio, pruebas y calidad.
- **Metodologías Ágiles:** Enfoques de desarrollo de software basados en principios como la entrega incremental, la colaboración con el cliente, la autoorganización de los equipos y la respuesta flexible a los cambios.

- **Scrum:** Marco de trabajo ágil para el desarrollo, entrega y mantenimiento de productos complejos, que se caracteriza por la división del trabajo en Sprints, roles definidos (Product Owner, Scrum Master, Equipo de Desarrollo) y ceremonias (Reunión de Planificación del Sprint, Reunión Diaria de Scrum, Revisión del Sprint, Retrospectiva del Sprint).